R-type (register-register)

```
31    28 27 26 25   22 21   18 17   14 13        0
+------+----+-------+------+-------+-----------+
|opcode|dtype|  rd   | rs1  | rs2  |   unused   |
+------+----+-------+------+-------+-----------+
 widths: 4      2    4      4      4       14
```

语义示例：`rd = op(rs1, rs2)`
适用：ADD, SUB, MUL, AND, OR, XOR, TDOT（Tensor）等。

I-type (immediate / memory / branch with rs1 + imm)

```
31    28 27 26 25   22 21   18 17            0
+------+----+-------+------+-------+------------+
|opcode|dtype|  rd   | rs1  | imm[17:0] (signed) |
+------+----+-------+------+-------+------------+
 widths:4      2    4      4        18
```

语义示例：

- `ADDI rd, rs1, imm`
- `LD rd, [rs1 + imm]` (imm is signed byte offset; we enforce 8-byte alignment or doc it)
- `ST rs2, [rs1 + imm]` (for store we place store value in rs2 via using rs1 field as base and rd as source, see encoding note below)
- `BRZ rs1, imm` (if rs1==0 PC += imm<<2 or PC = PC + imm_bytes) — we'll keep imm as number of bytes (or words); define in doc.

> 说明：18-bit signed immediate 给出 ±131k 范围（bytes），足够用于 loop offsets 与 memory offsets in this lab.

J-type (jump / long target)

```
31    28 27 26                      0
+------+----+-----------------------------+
|opcode|dtype|       target[27:0] (word aligned) |
+------+----+-----------------------------+
 widths:4      2           28
```

语义示例：`JUMP target` — target is absolute word address (we can left-shift/interpret as needed). We keep dtype field unused but reserved.

✅ Updated Opcode Table

Opcode	Instruction	Type	Execution Unit	Description
0000	NOP	R	—	No operation
0001	ADD	R	SIMD ALU	4-lane int16 vector add
0010	SUB	R	SIMD ALU	4-lane int16 vector subtract
0011	AND	R	SIMD ALU	4-lane bitwise AND
0100	OR	R	SIMD ALU	4-lane bitwise OR
0101	XOR	R	SIMD ALU	4-lane bitwise XOR
0110	MUL	R	SIMD ALU	4-lane int16 multiply
0111	RELU	R	SIMD ALU	4-lane ReLU activation
1000	ADDI	I	SIMD ALU	Vector + immediate
1001	LD64	I	LSU	Load 64-bit vector from memory
1010	ST64	I	LSU	Store 64-bit vector to memory
1011	BRZ	I	Control	Branch if zero
1100	BRNZ	I	Control	Branch if not zero
1101	JUMP	J	Control	Unconditional jump
1110	TDOT	R	Tensor Unit	BF16 vector dot product
1111	TDOT_RELU	R	Tensor Unit	BF16 dot product + fused ReLU